

Operator: `Conv2d`

Phase 1.5 nấc 2 → đầu vào cho Phase 2A (viết operator) và Phase 2D (golden model C).

1. Vai trò

Trượt một cửa sổ nhỏ (kernel) khắp ảnh, mỗi vị trí tính một tổng nhân-cộng. Cùng một bộ trọng số được **dùng lại** ở mọi vị trí — đó là điểm khác căn bản với `Linear`, và là lý do conv rẻ về bộ nhớ.

2. Công thức

Kích thước đầu ra

```
H_out = floor( (H_in + 2*padding - kernel) / stride ) + 1
W_out = floor( (W_in + 2*padding - kernel) / stride ) + 1
```

Nguồn: PyTorch docs, mục *Shape* của `torch.nn.Conv2d`.

Suy ra từ đâu: đếm số vị trí đặt được cửa sổ. Mép trái cửa sổ nhảy theo bước `stride`, điều kiện còn nằm trong ảnh là `mép_trái + kernel ≤ H_in + 2*padding`. Số vị trí = số bước hợp lệ **+ 1** (vì vị trí 0 cũng tính).

Hệ quả hay dùng: `padding = (kernel - 1) / 2` thì `H_out = H_in` (same padding). Chỉ ra số nguyên khi kernel **lẻ** — một trong các lý do kernel luôn lẻ.

kernel	padding giữ nguyên size
3	1
5	2
7	3

Số tham số

```
params = kernel_h × kernel_w × in_channels × out_channels + out_channels
                                             └ bias
```

Không có kích thước ảnh trong công thức. Chạy trên 28×28 hay 7×7 cũng chừng đó trọng số. Đây là câu trả lời cho "vì sao conv rẻ mà fc đắt".

Số phép tính (FLOPs)

```
MAC = kernel_h × kernel_w × in_ch × out_ch × H_out × W_out
```

Chỗ này **có** kích thước ảnh. Params quyết định bộ nhớ, MAC quyết định tốc độ/năng lượng — hai trục khác nhau, đừng lẫn.

3. Mã giả

Cho 1 ảnh (bỏ chiều batch). **Bản chính — đệm 0, luôn đủ K×K tap.** Đây là cách golden model C (`golden_c/golden_model.c`, hàm `conv`) đang làm và là cách hợp với FPGA (xem mục 5).

```
conv2d(input [IN_CH][IN_H][IN_W],
        weight[OUT_CH][IN_CH][K][K],
        bias [OUT_CH],
        output[OUT_CH][OUT_H][OUT_W]):

# ---- Bước 1: tạo ảnh đã đệm 0 ----
HP = IN_H + 2*PAD
WP = IN_W + 2*PAD

# khung toàn 0 (0 SAU chuẩn hoá, không phải nền)
padded[IN_CH][HP][WP] = 0

# chép ảnh vào giữa, chứa viền PAD ô
for ic = 0 .. IN_CH-1:
    for y = 0 .. IN_H-1:
        for x = 0 .. IN_W-1:
            padded[ic][y+PAD][x+PAD] = input[ic][y][x]

# ---- Bước 2: tích chập, không có if ----
for oc = 0 .. OUT_CH-1:
    for oy = 0 .. OUT_H-1:
        for ox = 0 .. OUT_W-1:

            # khởi tạo bằng bias, KHÔNG phải 0
            acc = bias[oc]

            # dồn mọi kênh vào cùng 1 acc
            for ic = 0 .. IN_CH-1:
                for ky = 0 .. K-1:
                    for kx = 0 .. K-1:
                        # luôn đủ K×K tap, ô viền nhân với 0
                        iy = oy*STRIDE + ky
                        ix = ox*STRIDE + kx
                        acc += padded[ic][iy][ix]
                               * weight[oc][ic][ky][kx]

            output[oc][oy][ox] = acc
```

****Bản tham khảo — kiểm biên bằng `if` (không cần mảng đệm, bỏ các tap ngoài ảnh; hợp CPU, không** dùng cho FPGA).** Bỏ Bước 1, thay phần trong vòng `kx` của Bước 2 (dòng `iy`, `ix`, `acc +=`) bằng đoạn dưới. Kết quả giống hệt bản chính vì cộng `0 × w` không đổi tổng:

```

iy = oy*STRIDE - PAD + ky
ix = ox*STRIDE - PAD + kx
# ngoài ảnh thì bỏ tap
if 0 <= iy < IN_H and 0 <= ix < IN_W:
    acc += input[ic][iy][ix]
        * weight[oc][ic][ky][kx]

```

6 vòng lặp lồng nhau. Vòng ngoài duyệt **vị trí đầu ra**, vòng trong duyệt **cửa sổ kernel**.

Ánh xạ toạ độ: điểm ra (oy, ox) dùng ô vào $iy = oy \cdot STRIDE - PAD + ky$ (tương tự cho ix). Trong bản chính, $-PAD$ biến mất vì ảnh đã được dịch vào $+PAD$ lúc chép sang `padded` — hai cái triệt tiêu nhau. Dấu $- PAD$ là chỗ dễ sai nhất.

Tính 1 điểm output — phần ruột của 6 vòng lặp

Cố định `oc, oy, ox` thì chỉ còn 3 vòng trong:

```

out[oc][oy][ox] = bias[oc] +  $\sum_{ic} \sum_{ky} \sum_{kx}$  in[ic][oy-PAD+ky][ox-PAD+kx] · w[oc][ic][ky][kx]

```

Quy ước: $in[ic][r][c] = 0$ khi r hoặc c nằm ngoài $0 \dots H-1$ (zero-padding). Đây là công thức **toán** — mọi cách cài đặt (PyTorch, bản `if`, bản đệm 0, FPGA) đều phải ra đúng số này. Trong bản đệm 0: $in[ic][oy-PAD+ky][ox-PAD+kx] = padded[ic][oy+ky][ox+kx]$ — $-PAD$ ở đây và $+PAD$ lúc chép vào `padded` triệt tiêu nhau, nên code không còn $-PAD$.

Lớp	Số MAC cho 1 điểm	Ghi chú
conv1	$1 \times 3 \times 3 = 9$	1 kênh vào
conv2	$8 \times 3 \times 3 = 72$	8 lát 3x3 trên 8 kênh vào
conv3	$16 \times 3 \times 3 = 144$	

Chỗ hay hiểu sai: mỗi kênh vào **không** cho ra một output riêng. Cả 72 tích của `conv2` dồn vào **một accumulator**, cộng thêm bias, ra **một số duy nhất**. Kênh ra khác (`oc` khác) dùng bộ kernel khác trên **cùng** cửa sổ vào.

Hình có số thật (sinh bằng `make_conv_figures.py`, khờp PyTorch):

- `figures/conv_1diem_conv1.png`: 9 pixel $\times$ 9 trọng số $\rightarrow$ cộng $\rightarrow$ + bias
- `figures/conv_1diem_conv2.png`: 8 kênh $\times$ 9 $\rightarrow$ 8 tổng con $\rightarrow$ cộng $\rightarrow$ + bias $\rightarrow$ 1 số
- `figures/conv_padding_goc.png`: điểm ở góc, 5/9 tap rơi vào vùng đệm 0

4. Áp dụng vào `SmallCNN`

Lớp	in $\rightarrow$ out	H_in	H_out	Params	MAC
conv1	1 $\rightarrow$ 8	28	$(28+2-3)/1+1 = 28$	$3 \cdot 3 \cdot 1 \cdot 8 + 8 = 80$	$72 \times 28^2 \approx 56,000$
conv2	8 $\rightarrow$ 16	14	14	$3 \cdot 3 \cdot 8 \cdot 16 + 16 = 1,168$	$1152 \times 14^2 \approx 226,000$

Lớp	in→out	H_in	H_out	Params	MAC
conv3	16→16	7	7	$3 \cdot 3 \cdot 16 \cdot 16 + 16 = \mathbf{2,320}$	$2304 \times 7^2 \approx 113,000$

Để ý conv1 ít hơn conv3 **29 lần** params nhưng chỉ ít hơn **2 lần** phép tính. Đúng minh họa params ≠ FLOPs.

5. Mức FPGA-friendly

✅ **Rất tốt.** Phép toán duy nhất là MAC (nhân + cộng tích lũy) — đúng thứ mà mảng PE trên FPGA sinh ra để làm. Không có chia, không có hàm siêu việt.

Kernel 3×3 đặc biệt thân thiện: chỉ cần 9 bộ nhân và line-buffer 2 hàng. Kernel 7×7 cần 49 bộ nhân và buffer 6 hàng.

if kiểm biên hay đệm 0? (Thầy góp ý 23/09)

Bản tham khảo ở mục 3 dùng if để bỏ các tap rơi vào vùng padding. Trên CPU cách đó tiết kiệm được phép nhân. **Trên FPGA thì ngược lại**, nên bản chính ở mục 3 và golden model C dùng đệm 0:

	Có if (bỏ tap ở biên)	Đệm 0, luôn đủ 9 tap
Số MAC conv / ảnh	351.008	395.136 (+11,2%)
Thời gian trên FPGA	Không nhanh hơn: 9 bộ nhân song song vẫn mất 1 chu kỳ, bỏ 5 thì 5 bộ ngồi không	1 chu kỳ / điểm, đều
Mạch thêm	Bộ so sánh $0 \leq i_y < H$, $0 \leq i_x < W$ cho từng tap + MUX → tốn LUT	Không. Line buffer đẩy số 0 ra ở biên
Luồng điều khiển	Không đều theo vị trí → pipeline và timing khó hơn	Mọi vị trí giống hệt nhau

Phân phí theo lớp (đo bằng inference_python.py, khớp tính tay):

Lớp	Đủ 9 tap	Bỏ padding	Phí
conv1 28×28	56.448	53.792	4,7%
conv2 14×14	225.792	204.800	9,3%
conv3 7×7	112.896	92.416	18,1%
Tổng	395.136	351.008	11,2%

Kết luận: giữ đủ 9 phép, đệm 0. Hai bản cho kết quả **giống hệt** vì cộng $0 \times w$ không đổi tổng. Con số **414.265 phép/ảnh** trong README.md đã đếm theo kiểu đủ 9 tap (395.136 MAC conv + 10.192 ReLU + 7.488 so sánh pool + 1.440 MAC fc + 9 so sánh argmax).

Bài học: đếm số phép là thước đo **phần mềm**. Phần cứng đo bằng **chu kỳ, diện tích (LUT/DSP), timing**. Giảm phép toán mà làm luồng điều khiển rẽ nhánh thì có thể lỗi.

6. Chú ý khi port sang C

Bẫy	Chi tiết
Thứ tự chiều	PyTorch: <code>weight[out_ch][in_ch][kH][kW]</code> . Keras: <code>[kH][kW][in_ch][out_ch]</code> — ngược hẳn . Lấy nhầm thứ tự thì không crash, chỉ ra kết quả sai
**Khởi tạo <code>acc</code> **	Bằng <code>bias[oc]</code> , không phải 0. Quên thì lệch đúng bằng bias, rất khó nhìn ra
Dấu của PAD	<code>iy = oy*STRIDE - PAD + ky</code> . Dấu trừ. Viết cộng thì ảnh bị dịch
Biên	Ngoài ảnh coi như 0, không phải lấp pixel mép
Giá trị đệm	Đệm 0 sau chuẩn hoá , không phải màu nền. Nền MNIST sau chuẩn hoá là <code>-0.4242</code> , khác 0. PyTorch đệm 0 sau <code>Normalize</code> nên C cũng phải đệm 0.0

7. Tự kiểm tra

Làm trên giấy, không chạy code:

- Với `conv1` của SmallCNN (`IN_CH=1`, `OUT_CH=8`, `K=3`, `PAD=1`, `STRIDE=1` , ảnh 28×28): vòng lặp trong cùng chạy **tổng cộng bao nhiêu lần**? So với con số MAC ở mục 4.
- Tính `output[0][0][0]` cần bao nhiêu phép nhân? Trong số đó bao nhiêu phép rơi vào vùng padding (bản `if` bỏ đi, bản đệm 0 nhân với 0)?
- Nếu đổi `PAD` từ 1 xuống 0, `H_out` của `conv1` thành bao nhiêu? Shape trước `flatten` đổi thế nào? `fc` còn bao nhiêu params?

Đáp án

Câu 1 — Vòng lặp trong cùng chạy bao nhiêu lần

$$\begin{aligned} \text{số lần lặp} &= \text{OUT_CH} \times \text{OUT_H} \times \text{OUT_W} \times \text{IN_CH} \times K \times K \\ &= 8 \times 28 \times 28 \times 1 \times 3 \times 3 \\ &= 56.448 \end{aligned}$$

Bằng **đúng** con số MAC ở mục 4. Nhưng hai con số đó **không cùng nghĩa**, và đây mới là phần đáng học:

	Giá trị
Số lần thân vòng lặp được vào	56.448
Số phép nhân thật sự chạy	53.792

	Giá trị
Số lần bị lệnh kiểm biên bỏ qua	2.656 (4,71%)

Chênh lệch là các vị trí rơi vào vùng padding. Công thức $k \times k \times in \times out \times H_{out} \times W_{out}$ ở mục 4 đếm cả những vị trí đó, nên nó là **cận trên**, không phải số phép nhân thật.

Kiểm chéo bằng cách đếm ngược — mỗi pixel đầu vào được bao nhiêu cửa sổ dùng tới:

```

pixel trong lòng ảnh (26×26 = 676) × 9 cửa sổ = 6.084
pixel mép không phải góc (104) × 6 cửa sổ = 624
pixel góc (4) × 4 cửa sổ = 16
tổng = 6.724 phép nhân / cặp (in_ch, out_ch)

6.724 × 1 × 8 = 53.792 ✓ khớp

```

Hệ quả cho FPGA: 2.656 phép "thừa" này **không** làm chậm phần cứng. 9 bộ nhân chạy song song, mỗi điểm output mất 1 chu kỳ dù có bao nhiêu tap là số 0. Bỏ chúng bằng `if` hay xử lý riêng viền ảnh chỉ thêm bộ so sánh và MUX. Vì vậy thiết kế theo **56.448** (đủ 9 tap, đệm 0) — xem mục 5.

Câu 2 — `output[0][0][0]`

Đây là góc trên trái, vị trí khắc nghiệt nhất. Cửa sổ 3×3 trải trên $ih, iw \in \{-1, 0, 1\}$:

```

      iw=-1  iw=0  iw=1
ih=-1  ✗     ✗     ✗      ✗ = ngoài ảnh (padding)
ih= 0   ✗     ✓     ✓      ✓ = pixel thật
ih= 1   ✗     ✓     ✓

```

9 vị trí cửa sổ, chỉ 4 phép nhân với pixel thật, 5 rơi vào padding. Trên CPU, 5 phép đó là thừa. Trên FPGA vẫn nhân đủ 9 (5 phép nhân với 0) vì 9 bộ nhân chạy song song, không tốn thêm chu kỳ.

Bốn góc đều như vậy (5 phép bỏ), các ô mép không phải góc bỏ 3 phép, ô trong lòng ảnh không bỏ phép nào. Cộng lại: $4 \times 5 + 104 \times 3 = 332$ phép bỏ cho mỗi cặp `(in_ch, out_ch)` — đúng con số ở Câu 1.

Câu 3 — Đổi `PAD` của `conv1` từ 1 xuống 0

```

H_out của conv1 = floor((28 + 2×0 - 3) / 1) + 1 = 26

```

Chuỗi shape sau đó (`conv2`, `conv3` vẫn `PAD=1`):

Bước	<code>PAD=1</code> (hiện tại)	<code>PAD=0</code> ở riêng <code>conv1</code>
conv1	8 × 28 × 28	8 × 26 × 26
pool	8 × 14 × 14	8 × 13 × 13
conv2	16 × 14 × 14	16 × 13 × 13
pool	16 × 7 × 7	16 × 6 × 6
conv3	16 × 7 × 7	16 × 6 × 6

Bước	PAD=1 (hiện tại)	PAD=0 ở riêng conv1
pool	$16 \times 3 \times 3$	$16 \times 3 \times 3$
flatten	144	144 — không đổi
** fc params**	1.450	1.450 — không đổi

Kết quả ngược trực giác: kích thước trung gian đổi ở cả 5 bước đầu, nhưng tới `flatten` thì quay về đúng 144, `fc` không đổi một tham số nào, tổng model vẫn 5.018.

Lý do: `floor` **không tuyến tính**. Qua ba lần pool, chênh lệch nhỏ ở đầu bị `floor` nuốt mất — 13 và 14 đều dẫn về cùng một kết quả sau khi tiếp tục chia đôi. Bài học: **không được suy diễn tỉ lệ input → output, phải tính lại từng bước**. Phần này đã ghi vào Word bước 4 mục 6.4.